

CLBIPP — Build Plan

Client / Vendor App first · then Field Agent · then Admin

Goal: ship a complete, installable PWA of the vendor-side journey from the approved wireframe — clean design, real auth, no shortcut vulnerabilities — in **2 weeks**. The lead then circulates it for feedback, so it must be stable, deployed on a shareable URL, and easy to file issues against.

Stack: Next.js (TypeScript, App Router) · Supabase (Postgres, Auth, Storage, Realtime) · Tailwind + shadcn/ui · Vercel. PWA via app manifest + service worker.

Build order: Vendor app first (this plan), then Field Agent, then Admin, in sequence.

How we've divided the work

Three lanes, split along natural seams in the codebase so the streams rarely block each other.

Person	Owns	Notes
Person A	Auth & live experience: Supabase Auth, session / route protection, access-control (RLS), login + signup screens, and the realtime tracking screens.	Owns the security layer and the live-data tracking flow end to end. Auth lands early so the others can gate their screens.
Person B	Data & records: schema + typed DB client, onboarding & KYC capture, the data-display screens (dashboard, compliance, profile), certificate generation (PDF), and the internal seed / simulation surface.	Owns the data model the whole team codes against, so it lands first. Onboarding writes the profile row, so it sits with the profile data.
Person C	Core flow & delivery: design-system component library from the wireframe, the full request → offer → handover flow, PWA + offline, deployment / CI.	Owns the shared component kit the other two build on, plus the main transaction journey end to end.

Scope guardrail for this sprint

- **Vendor side only** — the 17 wireframe screens. No field-agent or admin screens this sprint.
- The vendor app still needs *something* to produce offers and advance battery status (normally the agent/admin apps). We fake that with a tiny **internal seed + simulation surface** — a protected route + seed script, *not* a vendor screen. It's what we click during the demo to show live tracking move.
- No recovered value / recovery rate anywhere on the vendor side (per the wireframe decision). The offer carries a price + qualitative rationale only.

Phase 0 — Setup · half a day, all three together

A single call. Nothing else starts until every box here is green.

- ☐ Repo created, all three have access, `main` protected (PRs only, 1 review to merge).
 - ☐ Next.js + TypeScript scaffolded, Tailwind + shadcn/ui initialised, `npm run dev` works for everyone.
 - ☐ Supabase project created. `ANON_KEY` + `DATABASE_URL` shared securely (never committed). Service-role key lives only in Vercel env. — *Person B drives this*
 - ☐ Vercel linked to the repo so every PR gets a preview URL. — *Person C drives this*
 - ☐ Issue tracker seeded with the tasks below as tickets.
 - ☐ Agree the two contracts everything codes against: the **offer object shape** and the **status lifecycle** `requested → scheduled → collected → tested → processed → recovered → certified`. Write them in the tracker. Don't deep-code until these are locked.
-

Phase 1 — Foundations · Days 1–4

Mostly parallel, with two ordering rules (below). By the end, the app has a database, auth, and a component kit — the foundation the rest of the build depends on.

Ordering rules: (1) Person B's schema + types must land before A and C wire to them — B posts to the group chat the moment types are pushed. (2) Person C's core components (Button, Field, Card) should land early so the screen-builders aren't blocked. Until B's types land, A and C build against the agreed shapes with local mock data.

Person B — schema first, then data scaffolding

Do the schema first — it unblocks everyone, so it's the team's critical path on Day 1.

- ☐ 1. Author the schema as SQL migrations (versioned, not dashboard clicks): `profiles`, `pickups`, `offers`, `status_events`, `certificates`. Generate typed DB client; export shared types. *Post to the group chat when types are pushed — A and C are waiting.*
- ☐ 2. Validation schemas (Zod or equivalent) for every form payload — one source of truth for client + server checks.
- ☐ 3. Stub the dashboard + compliance screens against mock data so they're ready to wire once the flow produces records.

Order matters here: the schema is first because everything else depends on it.

Person A — auth, then access-control

Auth depends on the schema (you write a profile row on sign-up), so it follows B's types. Build login/signup against mock shapes first if B's types aren't in yet.

- ☐ 1. Wire Supabase Auth: email + password sign-up / sign-in, session via the Next.js server client. On sign-up, create the matching `profiles` row. (*screens: login, signup-type, signup-individual, signup-fleet*) *needs B's types*

- ❑ 2. Route-protection middleware: unauthenticated users reach only login / signup; everything else redirects. One public exception: the certificate verify link via a `public_token`.
- ❑ 3. Row-Level Security on every table: a vendor can read/write only rows where `vendor_id = auth.uid()`. Write them as SQL migrations (one policy per table). *needs B's schema*

Person C — design system from the wireframe

- ❑ Extract the wireframe's tokens (colours, type scale, radii, shadows) into Tailwind config + CSS variables so the app matches the wireframe exactly.
- ❑ Build the primitives the screens repeat: Button, Field/Input, Card, StatusBadge, Banner, ListRow, BottomTabBar, phone-frame AppShell. *Land Button/Field/Card first — everyone needs them.*
- ❑ Shared empty / loading / error state components so every screen handles them consistently.
- ❑ The lifecycle Timeline component (presentational, takes an array of stages) — used by the tracking screens later.

Phase 1 ends when: B has posted that types are live, A has posted that auth works, and C has posted that the core components are merged. Nobody starts Phase 2 work that depends on these until then.

Phase 2 — Core journey · Days 4–9

Each person builds their slice of the end-to-end flow on the now-stable foundations. Fully parallel — the dependencies were resolved in Phase 1.

Person A — realtime tracking & secure access

- ❑ Tracking screens wired to real data: in-progress / recovered / certified, reading from `status_events`. (*screens: track-progress, track-recovered, track-certified*)
- ❑ Supabase Realtime: a status change pushed from the sim surface updates the vendor's timeline live, no refresh. Subtle pulse on the active stage.
- ❑ Secure tracking link — token-based access to a single pickup's status without exposing other data.
- ❑ Profile screen + log out. (*screens: profile*)

Person B — onboarding, data-display screens, certificates & the sim surface

- ❑ Account-type selector + the two sign-up forms (Individual minimal; Fleet with GST / PAN / business address / EPR + KYC upload to Storage). Sits on A's auth (Phase 1, already done) and writes the `profiles` row. (*screens: signup-type, signup-individual, signup-fleet*)
- ❑ Dashboard — empty state and populated state (totals + recent-pickups list, each row routing to its tracking state). (*screens: dashboard-empty, dashboard*)
- ❑ Compliance log — all certificates with filter + export action. (*screens: compliance*)
- ❑ Certificate generation: render a styled EPR / recycling certificate to PDF, store in Supabase Storage, expose a download + a public verify token. (*screens: certificate*)

- ☐ The internal seed + simulation surface: seed script (demo vendors, pickups, offers, one fully-certified example) and one service-role-only endpoint to attach an offer / advance status. Clearly marked as throwaway, not a production feature. *Self-contained — this is what advances status during the demo.*

Person C — the request → offer → handover flow

- ☐ Request-pickup form: creates a pickups row at status requested; validation + submitted confirmation. (*screens: request, submitted*)
- ☐ Scheduled view — agent/ETA placeholder; reschedule / cancel while still pre-collection. (*screens: scheduled*)
- ☐ Offer screen — pathway + estimated price + qualitative “why this price” rationale (no value/rate). Accept → handover, status collected. (*screens: offer, offer-breakdown, handover*)

Phase 2 ends when: a vendor can sign up, request a pickup, get an offer, accept it, and — as B advances status from the sim surface — watch the timeline update live and download a certificate at the end.

Phase 3 — PWA, hardening & ship · Days 9–13

Turn the working app into an installable, deployed, feedback-ready product. Ownership noted per block.

PWA, offline & accessibility — Person C

- ☐ Web app manifest (name, icons, theme colour, standalone display) so it installs to home screen on Android / desktop and adds-to-home on iOS.
- ☐ Service worker: cache the app shell + static assets for offline launch; graceful offline banner for data that needs the network.
- ☐ Accessibility pass on shared components: focus states, labels, contrast, reduced-motion.
- ☐ Verify install + offline launch on a real phone; capture screenshots for the submission.

Security & data hardening — Person A (with B)

- ☐ Re-test every RLS policy with a second vendor account — confirm zero cross-vendor access. Document the test in the repo.
- ☐ Server-side validation on every input (never trust the client). File-upload type/size limits on KYC + certificate storage.
- ☐ Confirm no secrets in the client bundle; service-role key only on the server.

Quality, polish & feedback-readiness — all three

- ☐ Full click-through of the wireframe journey on a mobile viewport; fix any layout / nav mismatch against the wireframe.
- ☐ Lighthouse pass — aim green on PWA-installable + accessibility.
- ☐ Seed the deployed app with demo data so a reviewer landing cold sees a complete journey.

- ☐ Add a visible “Report an issue” link to the tracker (or a short form) so the lead can circulate it for feedback.
- ☐ Short README: live URL, test logins, what's real vs simulated, known gaps.

The demo path — definition of done

1. Install the PWA to home screen.
2. Sign up as a Fleet vendor (GST / KYC) → land on dashboard.
3. Request a pickup → see it go Requested → Scheduled.
4. View the estimated offer + rationale → accept → handover confirmed.
5. Open tracking; advance status from the sim surface → timeline updates live.
6. At Certified, download the EPR certificate PDF and view the compliance log.

If every step works on a phone from a cold start, the vendor app is done.

Risks & how we hold the 2 weeks

Risk	Mitigation
Scope creep from the other two apps	Hard line: vendor side only this sprint. Agent/admin behaviour is faked by the seed + sim surface.
Foundations slip and block everyone	B's types and C's core components are Day 1–4 priorities; owners post to the chat the moment they land. Others use mock data until then.
Auth/RLS done late = insecure demo	RLS is written as migrations in Phase 1 and re-tested in Phase 3 — never bolted on at the end.
Unconfirmed product details (fleet KYC fields, bulk-sell)	Build the structure with safe defaults; field-level specifics are cheap to change. Confirm async with the lead — don't block on it.

Open items to confirm with the lead (async, non-blocking): exact Fleet KYC document set; whether pricing differs by account type; whether bulk-sell is a distinct flow or just repeated requests; whether Fleet needs verification before transacting. None of these block Phase 0–1.

Ground rules

- No direct pushes to `main` — branch → PR → one review → merge.
- Daily 10-minute standup, voice, no agenda.
- Blocked > 2 hours → say so in the chat immediately.
- Finished something others wait on → say so explicitly. Don't assume commits get noticed.
- The two contracts (offer shape, status lifecycle) are locked in the tracker before deep coding.
- Last two days are for fixing, not new features.